SKYLINE WEATHER

使用者介面設計
Midterm Project · 2026

FROM BAD UI AUDIT TO A WORKING PRODUCT

Skyline Weather

把天氣講清楚，也把選擇權還給使用者。

從 weather.gov 的 8 項 Bad UI 問題與 9 張 Redesign 分鏡出發， 以 Django、Neon PostgreSQL 與 Render 實作可操作的前台、會員系統與管理後台。

組別	組員	繳交日期
第 8 組	組員：_____（學號：_____）	2026.11.01



介面設計期中專題 · **2026.11.01** · 5 pages / 8 models / 12 tests

不是重做一個天氣網站，而是把一份調查變成可操作的產品。

Same data, clearer decisions. 從「看懂天氣」到「知道下一步」。

5
front-end pages
Django + Neon

5 pages
前台主要頁面
今天／城市／詳情／警報／會員

12 cities
城市資料
含 8 則生效警報與 7 日預報

8 models
資料模型
前台查詢與後台管理共用

12 tests
自動化測試
頁面、查詢、會員、後台

五個頁面，對應五種任務

3–5 pages requirement

01	**今天**	現在天氣、12 小時趨勢、可行動建議、7 日	/
02	**城市總覽**	關鍵字、區域、天氣、溫度、警報、排序	/locations/
03	**城市詳情**	24 小時、7 日、觀測細節、此城市警報	/locations/<slug>/
04	**警報中心**	嚴重度、類型、區域、建議行動、時間軸	/alerts/
05	**會員中心**	收藏、查詢紀錄、預設城市、通知門檻	/dashboard/

TECH STACK

設計系統直接長成程式碼

沿用 Bad UI 作業的 Design Tokens（#1567D3、#0F2438、#35B6E8、溫度色、AQI 色），以 Django Template + 原生 CSS 實作，不依賴前端框架。

Django 6.1 · Neon PostgreSQL · Render · Gunicorn · Whitenoise · Selenium

12
城市 × 7 日預報

360
逐時預報資料

100%
測試通過

01 Audit
weather.gov 8 項問題

02 Design
9 張分鏡 + Design Tokens

03 Build
Django + Neon + Render

起點不是「做一個網站」，而是解決 8 個被量化的問題。

課程要求：5 頁前台、登入前後差異、多元查詢、Django 後台、Neon、雲端部署與 10 組 AI Prompt。

8
Bad UI issues
→ 8 design decisions

從 homework 帶進來的三個限制

1. **資訊層級不能只是「變漂亮」**
   原首頁 113 個連結、3 個搜尋框；新首頁只保留現在、建議、預報、警報四件事。
2. **同一個資料，前台與後台都要能用**
   不是做一個靜態 mockup：城市、預報、警報、會員都進資料庫，後台可編修。
3. **設計決策要能對回 Persona**
   通勤族需要 30 秒內做決定；戶外活動者需要時段建議；緊急人員需要影響範圍與行動。

單一搜尋入口
Ctrl K / 城市建議

可行動摘要
帶傘、最佳時段、AQI

登入個人化
收藏、預設城市、通知

可管理後台
8 個模型、搜尋篩選

113
首頁連結數
原始 weather.gov 首頁；使用者必須自己判斷哪些連結才有用。

77%
文字 ≤ 12px
首頁 77%、預報頁 75%；在 1280px 視窗下閱讀負擔高。

67 / 40
Lighthouse 分數
無障礙 67、SEO 40；成為本期把對比、標題層級與語意結構做好的依據。

課程原文：「完成可操作的前端網站、Django 管理後台與 Neon PostgreSQL 資料庫，並將前後台部署至雲端。」

- ✓ **5 頁前台**
  今天／城市／詳情／警報／會員
- ✓ **登入前後差異**
  收藏、預設城市、通知門檻
- ✓ **多元查詢**
  6 種城市條件 + 4 種警報條件
- ✓ **Django 管理後台**
  8 模型、搜尋、篩選、批次動作
- ✓ **Neon PostgreSQL**
  前後台共用同一份資料
- ✓ **雲端部署**
  Render + health check + 測試帳號
- ✓ **10 組 AI Prompt**
  前期 8 + 本期 2，含成果對照
- ✓ **期中報告 PDF**
  8 大段落 + 實際網站截圖

8 個問題，對應 8 個設計決策。

前期以 6 張原始截圖、量化統計與 Lighthouse 建立問題清單；本期把每一項變成畫面與程式。

8
issues
6 annotated shots

1 資訊層級失衡
113 個連結、3 個搜尋框搶注意力。→ **任務導向首頁、單一搜尋。**

2 搜尋與導覽不一致
首頁與預報頁各有不同搜尋樣式。→ **全站一致搜尋與 tabs。**

3 視覺語言分裂
多套標題、按鈕、連結與表格樣式。→ **Design Tokens 與元件。**

4 字級過小、密度過高
77% 文字 ≤ 12px，資訊牆難以掃讀。→ **內文 14px、輔助 12.5px。**

5 預報難以行動
大量文字與術語，缺少「所以呢？」→ **可行動摘要與最佳時段。**

6 技術術語與版本切換
KML、AFD、Versions 1–50 對一般使用者無意義。→ **分層揭露、白話優先。**

7 警報與地點脫節
Active Alerts 沒有影響區域與建議行動。→ **警報中心 + 行動指南。**

8 裝飾元素搶焦點
靜態地圖與公告佔據首屏。→ **首屏任務優先，公告降級。**

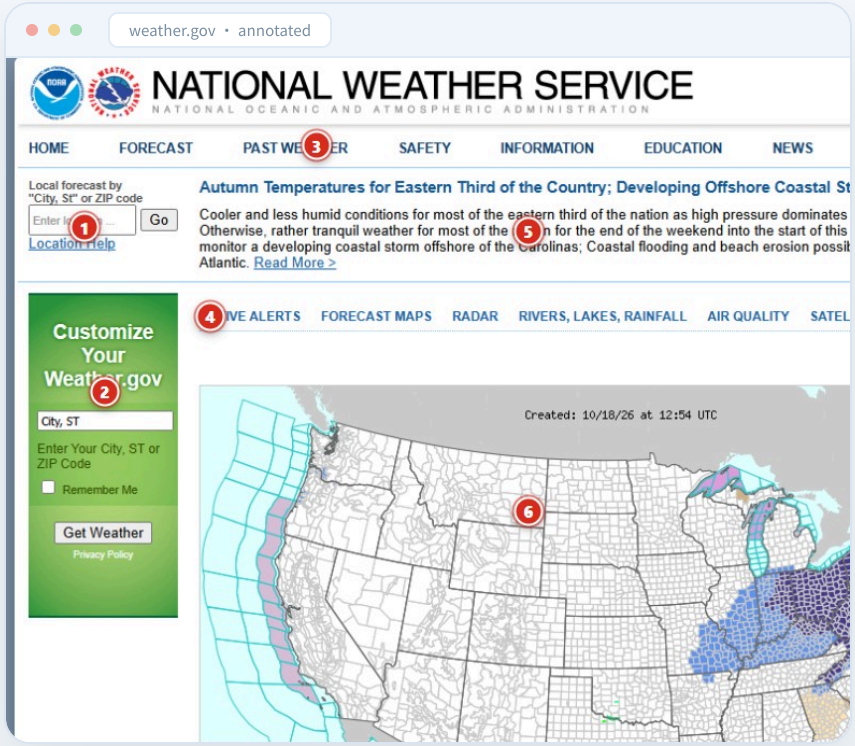


圖 1 前期報告的標註圖（節錄）。完整 6 張標註與 8 項問題見 homework 報告。

量化證據

113	77%	67	40
links	≤12px	a11y	SEO

改善目的：讓每一頁都回答一個問題。

五個頁面不是五個展示畫面，而是一條從「現在」到「接下來」再到「需要行動」的決策路徑。

4
design principles
5 pages

01
任務優先
首屏先回答現在天氣與是否需要注意，再把預報細節往下放。

02
一致的語言
同一套色彩、字級、按鈕與導覽；前台與後台都使用相同的資料命名。

03
可行動
每個數字旁邊都給出「所以呢」：帶傘、最佳時段、警報行動。

04
可及性
內文 14px、輔助 12.5px、對比 ≥4.5:1、焦點樣式與 skip link。

頁面	對應的 Bad UI 問題	主要功能	登入前後差異
今天 /	資訊層級、密度、裝飾搶焦點	即時觀測、12 小時趨勢、可行動建議、3 日與 7 日、警報摘要	訪客看 New York；會員看預設城市
城市總覽 /locations/	搜尋不一致、113 個連結	關鍵字、區域、天氣、溫度區間、有無警報、排序、分頁	訪客可查；會員留下查詢紀錄
城市詳情 /locations/<slug>/	預報難掃讀、術語過多	24 小時、7 日、觀測細節、此城市警報、鄰近城市	收藏按鈕；登入後可加入收藏
警報中心 /alerts/	警報與地點脫節、缺行動	嚴重度／類型／區域／關鍵字、時間軸、影響範圍、建議行動	訂閱通知需登入；訪客可瀏覽
會員中心 /dashboard/	缺少個人化與回饋	收藏、最近查詢、收藏城市警報、°F/°C、通知門檻	登入限定；管理員可進後台

單一搜尋：Ctrl K　12 小時趨勢　最佳外出 3 小時　AQI 分級　警報嚴重度排序　收藏與通知門檻

一個 Django 專案，前台與後台共用同一份資料。

Neon 是唯一的資料來源；Django ORM 同時服務前台查詢、會員功能與管理後台。

1
database
2 surfaces

CLIENT

瀏覽器

- Django Template + CSS + JS
- 5 個頁面、RWD、無框架
- 搜尋建議 / 單位切換

HTTPS
Render

RENDER

Django 6.1

- Gunicorn + Whitenoise
- Views / Templates / Forms
- Auth + Session + Messages
- Django Admin（後台）

ORM / psycopg
DATABASE_URL

NEON

PostgreSQL

- 8 個資料模型
- 會員、城市、預報、警報
- 後台修改即時反映前台

GitHub → Render push 後自動 build；`build.sh` 安裝套件、collectstatic、migrate、種子資料（僅空資料庫）。

Admin → Django 教師以管理員帳號登入 `/admin/`，直接新增城市、修改預報、啟停警報。

Health check `/healthz/` 執行 `SELECT 1`，同時確認 Django 與 Neon 都活著。

Request	View	ORM	Neon	Template	Response
瀏覽器送出 URL / 查詢參數	權限、篩選、Paginator、溫度單位	Location / Forecast / Alert	回傳資料列	SVG icon、sparkline、卡片	HTML + CSS + JS

DEPLOY PIPELINE • GITHUB → RENDER

Push GitHub main › **Build** pip install › **Collect** Whitenoise › **Migrate** Neon schema › **Seed** if empty › **Live** healthz

```
# Render environment
DEBUG=False
DATABASE_URL=postgresql://...neon.tech/neondb?sslmode=require
ALLOWED_HOSTS=.onrender.com
CSRF_TRUSTED_ORIGINS=https://*.onrender.com
```

Why Django Template?
不用 SPA build；表單、權限、SEO 與部署一次處理，適合兩週內交付。

Why Neon?
雲端 PostgreSQL、免費方案、連線字串可直接放 Render 環境變數。

Why Render?
GitHub 自動部署、health check、環境變數與免費 Demo 一次到位。

8 個模型，支撐 5 個頁面與 1 個後台。

資料模型先對齊 UI 需要的欄位，再讓 Django Admin 直接管理；前台不需要另外寫 API。

9
tables incl. User
Neon PostgreSQL

使用者
django.contrib.auth.User
- **username** / email
- password / is_staff
- last_login

會員設定
Profile • 1–1 User
- **unit** °F / °C
- **home_location** FK
- notify_app / email / sms
- min_severity

收藏
Favorite • N–N
- **user** FK
- **location** FK
- unique(user, location)

查詢紀錄
SearchHistory
- **query** / region
- results_count
- created_at

城市
Location
- **name / state / slug**
- region / county / ZIP
- lat / lon / timezone / station

即時觀測
CurrentConditions • 1–1
- **temp_f / feels_like_f**
- condition / humidity / wind
- AQI / UV / visibility

每日 / 逐時
DailyForecast • HourlyForecast
- **date / time**
- high_f / low_f / temp_f
- precip_percent / wind

警報
Alert
- **severity / type / urgency**
- starts_at / ends_at / areas
- description / instruction / source

Location 1–1 **CurrentConditions** · **Location** 1–N **Daily / Hourly / Alert** · **User** 1–1 **Profile** · **User** 1–N **Favorite / SearchHistory** · **Profile** N–1 **Location**（預設城市）

查詢不 hard-code
溫度、天氣、警報都由 ORM filter 組合；新增城市不用改 view。

後台所見即前台所得
Django Admin 直接操作同一組 models；修改後重新整理即可看到。

種子資料可重建
`seed_demo` 以 update_or_create + 重建預報，讓 demo 資料維持在「本週」。

MODEL → PAGE

Location → 首頁／查詢 · **Daily → 7 日** · **Hourly → 24 小時** · **Alert → 警報中心**

Profile／Favorite／SearchHistory → 會員

SEEDED DATA • NEON

12	84	360	8
locations	daily	hourly	alerts

首頁：把「現在該做什麼」放在第一屏。

訪客預設 New York；登入後自動切換到會員設定的預設城市與 °F/°C。

/
Today
first screen

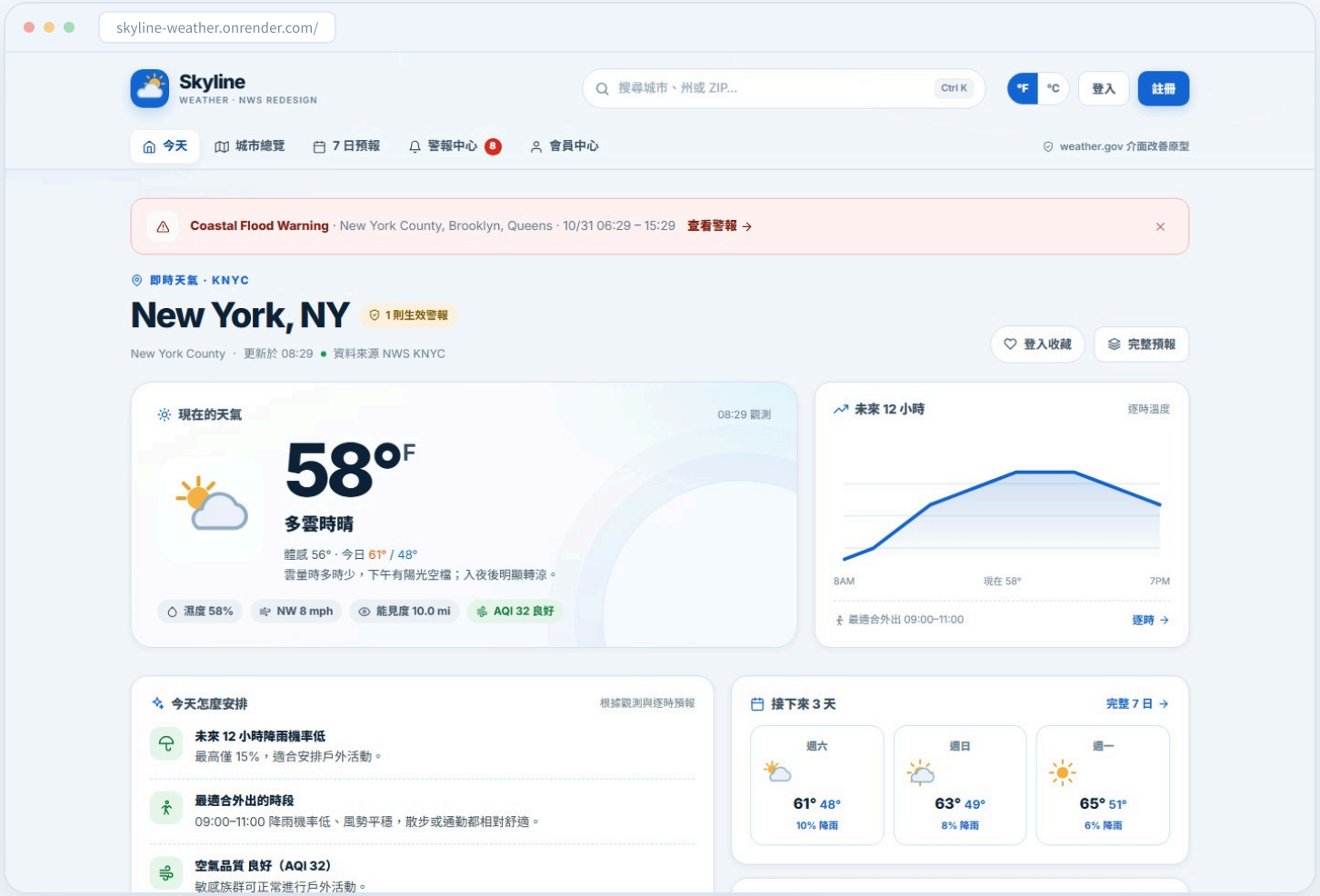


圖 2　首頁實際畫面（1440px 視窗，Selenium 擷取）。包含警報、現在天氣、12 小時趨勢、可行動建議與 3 日預報。

1　警報 strip：先講最嚴重的事
只顯示與目前城市相關的最高嚴重度警報；可關閉，不永久佔用版面。

2　Now card：溫度＋體感＋高低溫
大字溫度、SVG 天氣圖示、濕度／風／能見度／AQI 一次看到，避免來回切頁。

3　可行動建議：回答「所以呢」
降雨機率、最佳外出 3 小時、AQI 分級與風勢，直接給出行動建議。

4　7 日：週末標記＋降雨長條
高低溫、降雨機率與風勢同一個視線高度；週末以藍框標記。

設計對應
原網站 113 個連結 → 首頁 4 個任務區塊；原本需要 3 次點擊的天氣判斷，現在第一屏完成。

113→4
連結 → 任務區塊

3→1
搜尋框 → 單一入口

12→14
最小字級 (px)

城市總覽：把 6 種查詢條件放在同一張表單。

關鍵字、區域、天氣、溫度區間、有無警報、排序；訪客可用，會員的查詢會寫入紀錄。

12
cities
9 per page

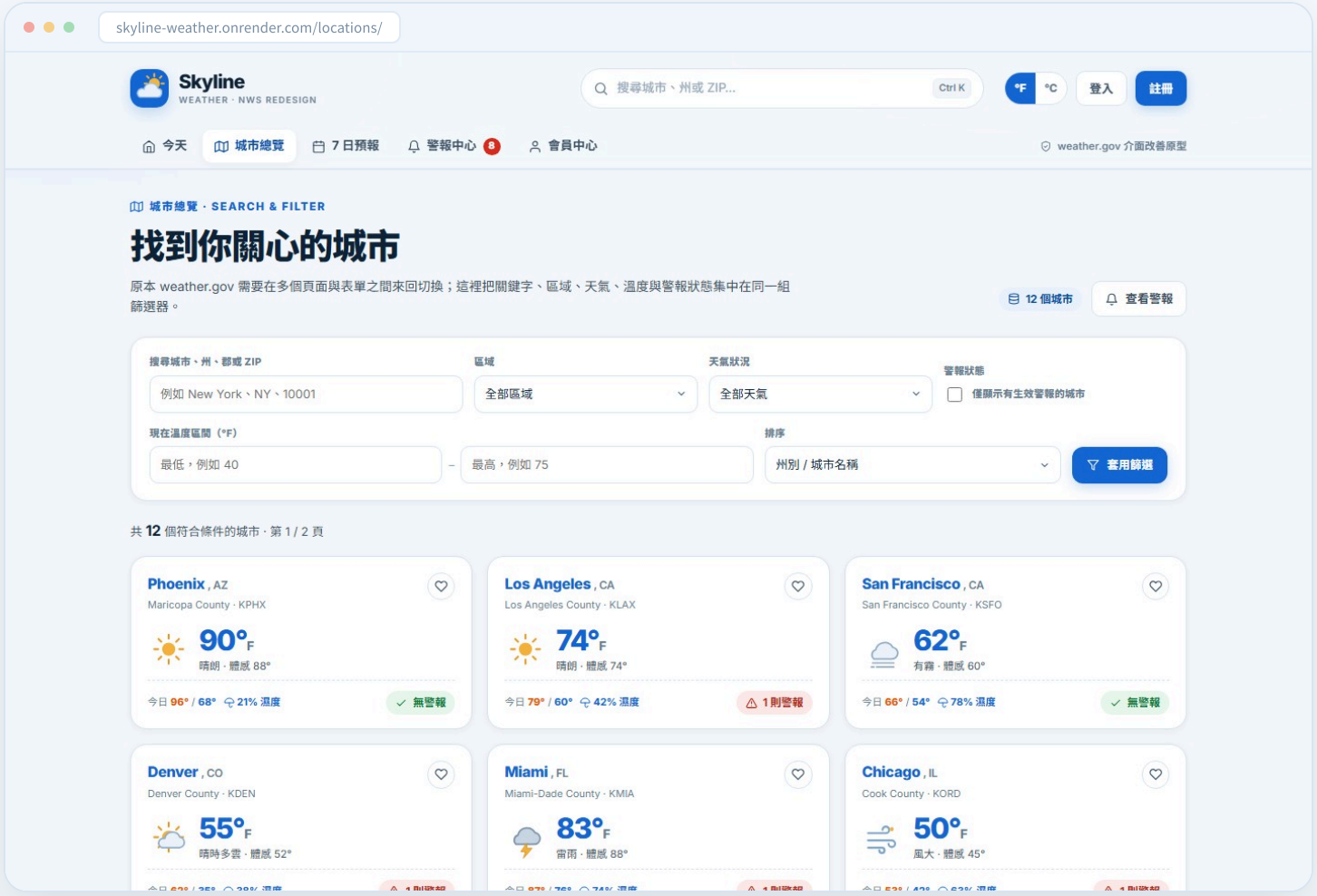


圖 3　城市總覽：篩選器、結果數量、排序、卡片與分頁。卡片直接顯示現在溫度、體感、今日高低溫與警報數。

QUERY DIMENSIONS

關鍵字	城市／州／郡／ZIP
區域	東北／東南／中西部…
天氣	晴／雨／雷雨／雪…
溫度	°F 區間 min–max
警報	僅顯示有生效警報
排序	州別／溫度／警報數

EXAMPLE

```
/locations/?q=New+York&region=northeast&condition=partly_cloudy&temp_min=50&temp_max=70&alerts=1
```

符合 1 筆　保留 query string　paginator

1. **訪客可直接查詢**
   不需要登入；登入後同一組 query 會寫入 SearchHistory，會員中心可一鍵重查。
2. **空結果也有下一步**
   顯示重設篩選 CTA，不讓使用者卡在空白頁。

城市詳情：一頁完成「現在、接下來、要注意」。

24 小時逐時、7 日、觀測細節與此城市警報集中在同一頁，減少來回切換。

24h
hourly
7-day

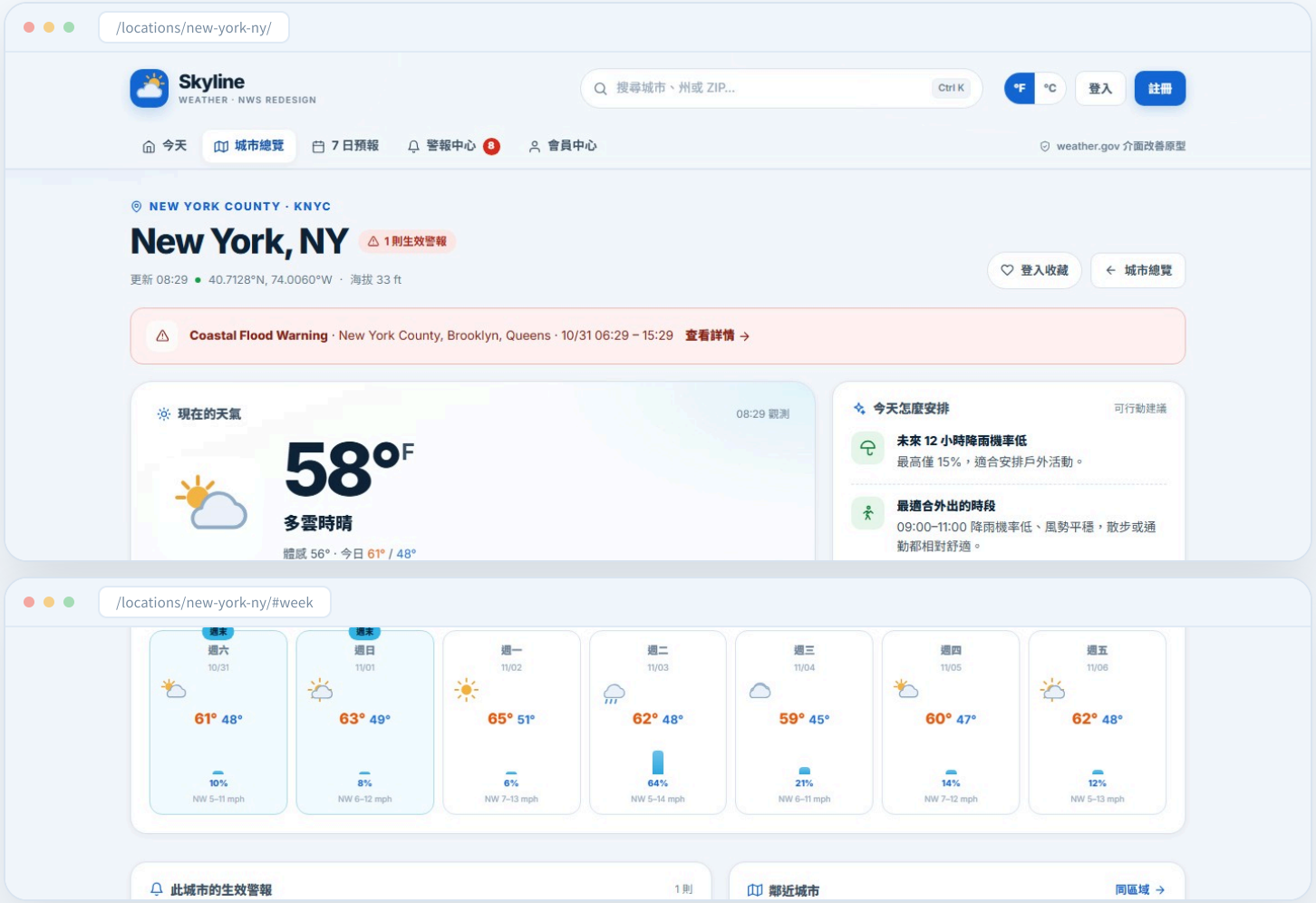


圖 4　同一頁的兩個區段：現在／逐時與 7 日；捲動即可比較。

觀測細節：不下判斷，先給原始資訊

體感 **56°F**	能見度 **10.0 mi**
紫外線 **3 中等**	AQI **32 良好**

1. **24 小時：水平時間軸**
   每個小時都有圖示、溫度、降雨機率與風速；第一格標記「現在」。
2. **7 日：週末與降雨並列**
   週末以藍框標記；降雨長條與百分比同時呈現，不靠顏色猜測。
3. **城市警報 + 鄰近城市**
   把「這個城市現在要注意什麼」與「附近天氣」放在同一段。

`24 hourly rows` `7 daily rows` `alert count` `favorite`

Before → After

原本：首頁 → 搜尋 → 預報 → 細節，至少 4 個步驟。現在：一頁同時看到現在、24 小時、7 日與警報。

警報中心：從「一堆警示」變成「可行動指南」。

依嚴重度排序，支援嚴重度、類型、區域與關鍵字；每一則都有影響區域、時間軸與建議行動。

8
active alerts
2 severe



圖 5　警報中心：統計、篩選、嚴重度色條、類型圖示與影響區域；右側是等級說明與訂閱 CTA。

2
Severe

5
Moderate

1
Minor

1. **先分級，再過濾**
   嚴重度色條＋排序；使用者不用自己判斷哪一則最重要。
2. **影響區域與時間**
   城市、郡、具體區域與有效時段；不再只有一個 Active Alerts 連結。
3. **建議行動**
   詳情頁把 NWS 指引轉成編號步驟，並提供「加入收藏」與通知設定。
4. **會員通知門檻**
   App／Email／SMS 與最低嚴重度；從「看警報」走到「被通知」。

設計對應：原網站問題 7
警報與地點脫節 → 每則警報都有主要城市、影響區域與行動連結。

查詢參數也可分享
`?severity=severe&region=northeast` 可直接篩出「東北部嚴重警報」，報告與 demo 都能重現。

登入前後：同一份資料，不同的個人化層次。

訪客完成公開查詢；會員多一層「記得我」：預設城市、收藏、單位、查詢紀錄與通知門檻。

2
states
guest / member

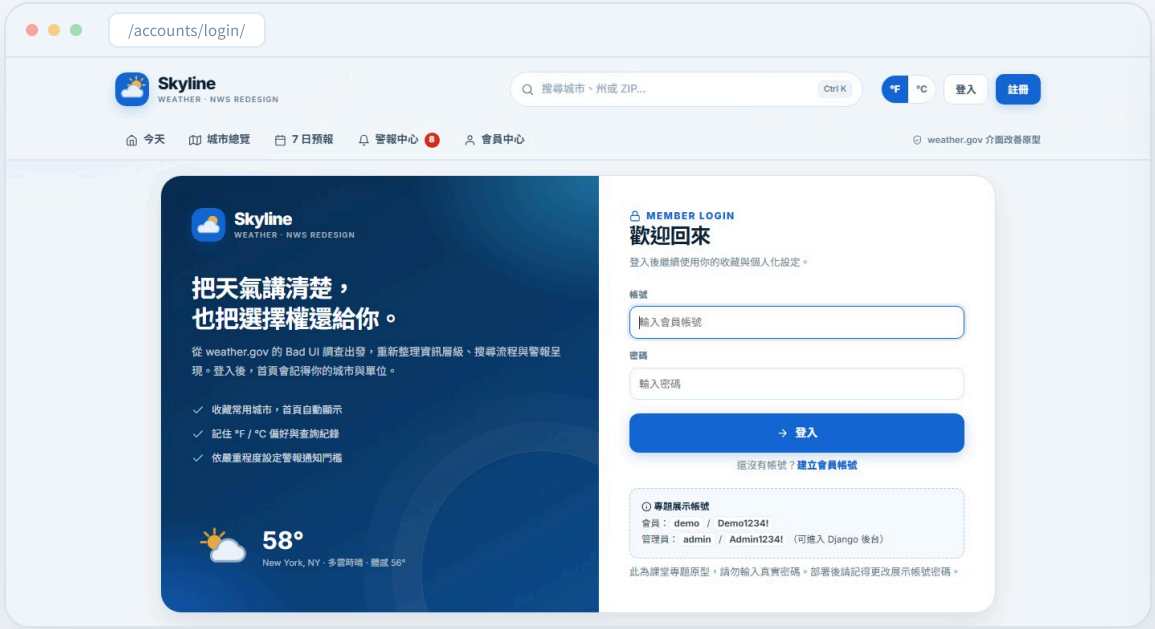


圖 6　登入頁：左側品牌與價值主張，右側表單與展示帳號；錯誤訊息與焦點樣式完整。

訪客：可查詢　收藏 → 引導登入　無查詢紀錄

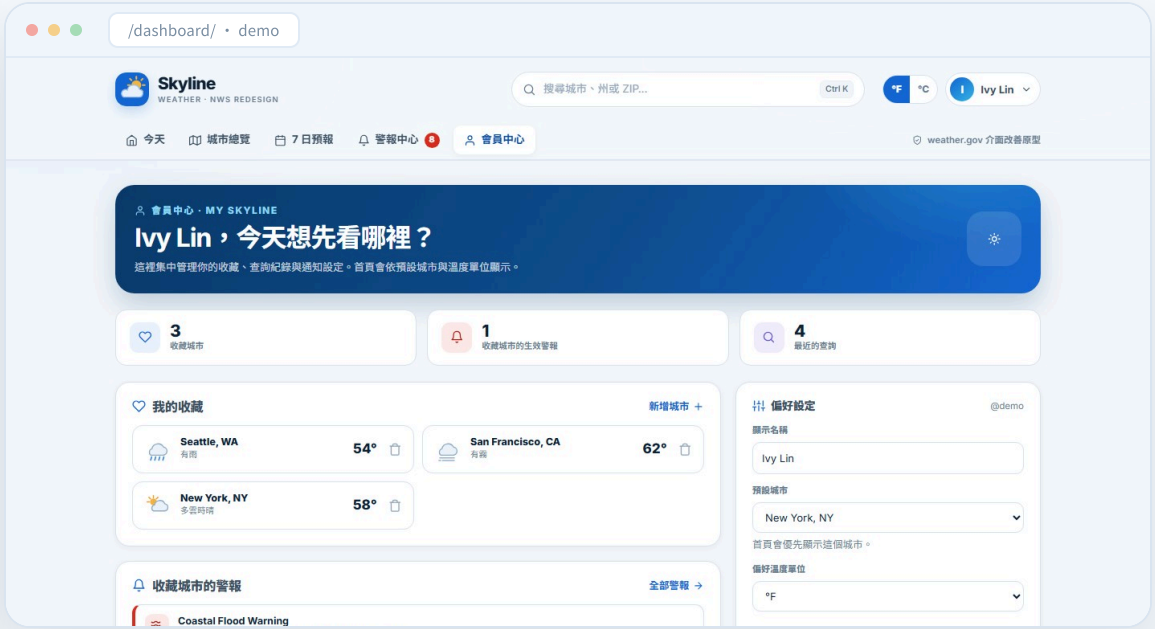


圖 7　會員中心：3 個收藏、1 則收藏城市警報、4 筆查詢、偏好設定與通知開關。

收藏 3　預設 New York　°F / °C　通知門檻

首頁記住城市
Profile.home_location；登入後首頁自動切換。

收藏與移除
Favorite unique(user, location)；POST + next 安全導回。

查詢紀錄
SearchHistory 只記登入者；會員中心一鍵重查。

通知偏好
App／Email／SMS + 最低嚴重度；與警報中心連動。

安全與權限　`@login_required` 保護會員頁　• POST + CSRF 收藏切換　• `next` 重新導向主機驗證　• 密碼使用 Django hash　admin 專屬後台

後台不是附件，是網站資料的來源。

8 個模型全部註冊到 Django Admin；城市、觀測、預報、警報、會員與收藏都能被搜尋、篩選與修改。

8
models
same Neon

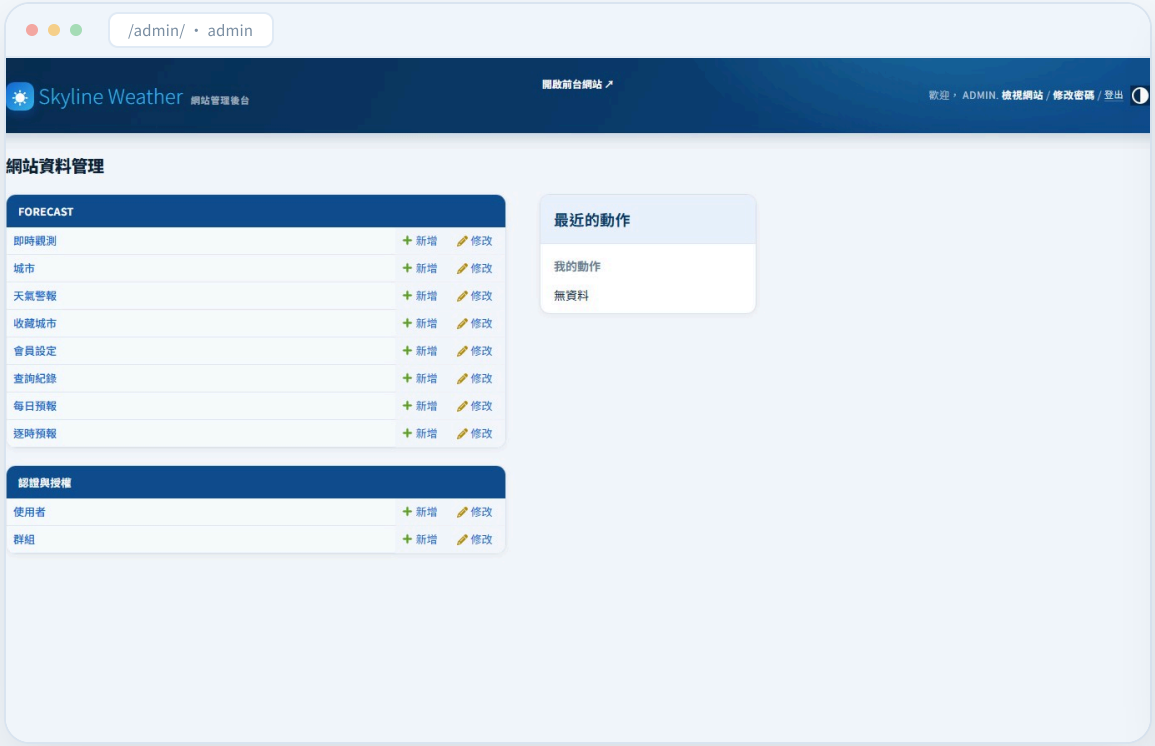


圖 8　自訂品牌後的 Django Admin：城市、即時觀測、天氣警報、收藏、會員設定、查詢紀錄、每日／逐時預報。

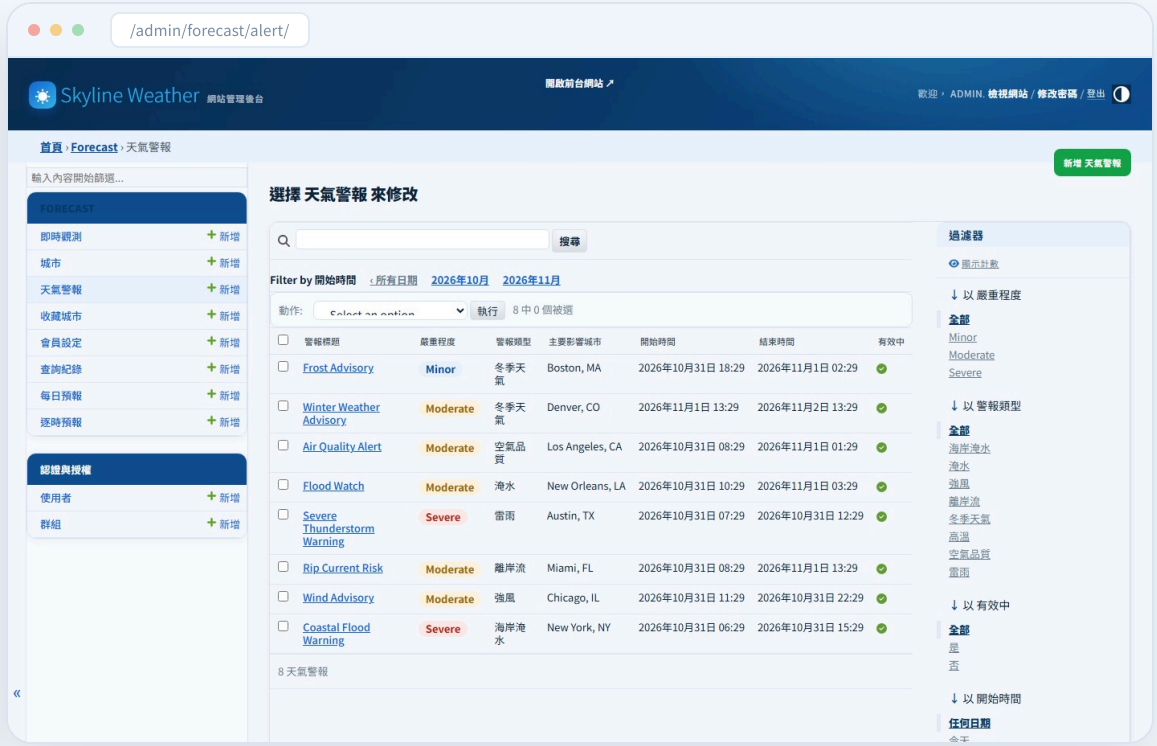


圖 9　警報管理：嚴重度 badge、類型、城市、起訖時間、有效狀態與批次啟停 action。

搜尋 × 篩選
每個 model 都有 search_fields 與 list_filter。

Inline 編輯
城市頁可直接編即時觀測、7 日預報與警報。

批次動作
警報可一次標記有效／失效，適合 demo 前整理。

同一份 Neon
後台修改後，重新整理前台即更新，無需同步腳本。

10 組 Prompt，從調查一路走到可操作網站。

前期 8 組完成 Bad UI 調查與 Redesign；本期 2 組把規範讀進來，並把設計落地成 Django 全端專案。

10
prompts
8 + 2

#	階段	使用目的	AI 回覆與實際成果
1	調查	定義作業範圍與交付項目	8 章節報告架構、分析／Persona／Flow／Redesign 清單
2	調查	擷取 weather.gov 6 個操作畫面	首頁、搜尋、預報、細節、AFD、Active Alerts 截圖
3	調查	量化連結數、字級與 Lighthouse	113 links、77% ≤12px、a11y 67、SEO 40
4	調查	找出並標註至少 5 項 Bad UI	8 項問題、6 張標註圖、對應改善方向
5	設計	建立 3 類 Persona / Story / Scenario	通勤族 Ivy、戶外 Michael、緊急管理 Carla
6	設計	設計 3 條 User Flow	Flow A 查現在／Flow B 規劃週末／Flow C 處理警報
7	設計	建立 Design System 與 9 張分鏡	Tokens + A1–A3、B1–B3、C1–C3；本期直接沿用
8	設計	整理前期報告並輸出 PDF	38 頁 PDF：分析、Persona、Flow、分鏡、Prompt
9	實作	閱讀期中規範與交付要求	5 頁、登入差異、後台、Neon、部署、10 Prompt
10	實作	把 homework 實作成 Django 網站	5 頁前台、8 模型、12 城市、8 警報、12 測試、部署檔

01–04 **調查 Investigation**
截圖、量化、8 項問題

05–08 **設計 Design**
Persona、Flow、Design System

09–10 **實作 Implementation**
Django、Neon、後台、部署

截圖索引： 10 組 Prompt 的實際 OpenCode 對話截圖請依規範附於報告附錄（前期 1–8 於 2026-10-18；本期 9–10 於 2026-10-31）。完整 Prompt 文字與成果對照見 **docs/AI_PROMPTS.md** 。

Prompt 不是「幫我寫」，而是把設計決策說清楚。

本期兩個 Prompt 分別定義「要做什麼」與「用什麼既有設計來做」；AI 產出後仍由我們檢查資料模型、畫面與測試。

2
this sprint
9–10

PROMPT 9 2026-10-31

「幫我讀一下這份文件，協助我完成介面設計的期中專題」

Attached: 期中專題.pptx

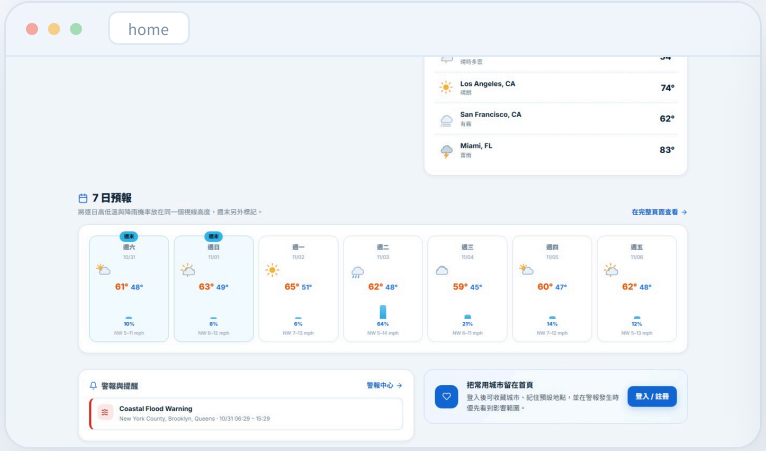
AI 回覆：整理 10 頁規範、5 頁前台、登入前後差異、後台／Neon、部署、10 組 Prompt、評分 30/70 與截止日；轉成需求對照表。

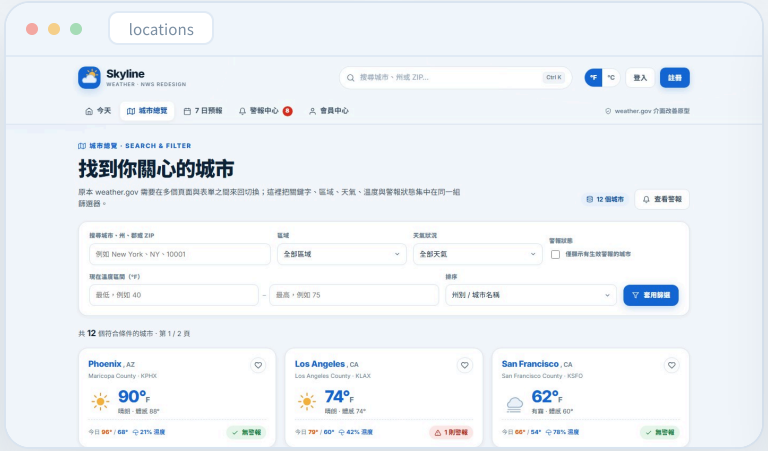
PROMPT 10 2026-10-31

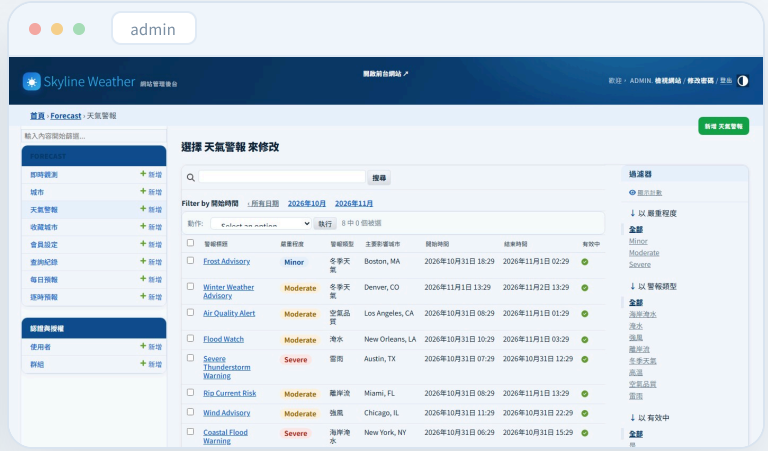
「之前做的 badui homework C:\Users\user\Desktop\bad-ui-homework」

延續 weather.gov 調查與 9 張分鏡

AI 回覆：讀取 homework 的截圖、Redesign HTML 與報告，建立 Skyline Weather：5 頁、8 模型、12 城市、會員、後台、Render 部署與 12 個測試。







AI 強項
把規範轉清單、產生初稿、批次檢查與截圖。

人工檢查
資料模型、篩選邏輯、RWD、空結果與部署安全性。

可重現
seed_demo 重建資料；測試 12/12 通過；截圖由 Selenium 產生。

部署與測試：從本機到可驗收的連結。

Django + Gunicorn + Whitenoise 部署到 Render；Neon 提供 PostgreSQL；`build.sh` 自動化建置與種子資料。

12/12
tests passed
healthz ok

FRONT-END
https://skyline-weather-gqoc.onrender.com/

DJANGO ADMIN
https://skyline-weather-gqoc.onrender.com/admin/

GITHUB REPOSITORY
https://github.com/0x4b4149/UI_Design_Mid

✓ **Neon PostgreSQL 連線**
DATABASE_URL + sslmode=require；前台與後台共用。

✓ **Render 建置與啟動**
bash build.sh → collectstatic → migrate → seed_demo --if-empty。

✓ **健康檢查**
`/healthz/` 執行 SELECT 1，回傳 database: up。

✓ **教師測試帳號**
前台 demo / Demo1234!；後台 admin / Admin1234!。

自動化測試 — 12 / 12 pass

公開頁面與 404	PASS
城市搜尋／6 種篩選	PASS
警報篩選與排序	PASS
搜尋 API / 分頁	PASS
登入保護與會員偏好	PASS
收藏切換 / °F↔°C	PASS
註冊建立 Profile	PASS
管理員後台 / healthz	PASS

測試方式
`python manage.py test forecast -v 2`；以 Django TestCase 建立城市／預報／警報／會員 fixture，涵蓋 GET/POST 與權限。

Gunicorn · Whitenoise · Render Blueprint · Neon pooled · free tier cold start

DEPLOY PIPELINE — deployed

1 • GitHub (push main) › 2 • Build (pip install -r) › 3 • Static (collectstatic) › 4 • Neon (migrate) › 5 • Seed (--if-empty) › 6 • Health (/healthz/)

```
# Render build log (excerpt)
Applying forecast.0001_initial... OK
Seeded 12 locations, 84 daily forecasts, 360 hourly, 8 alerts.
Booting worker with pid ... ✓
```

Demo 驗收路徑
首頁 → 城市總覽查 Seattle → 登入 demo → 收藏 → 警報中心 Severe → 後台修改警報 → 前台重新整理確認。

學到的不只是 Django，而是把設計決策落地的方法。

從「介面很亂」到「8 個問題、5 個頁面、8 個模型、12 個測試」；每一步都要能對回使用者與資料。

3
takeaways
next steps

8→5
問題 → 頁面
8 項 Bad UI 問題被收斂成 5 個任務導向頁面，不再用「功能清單」思考。

9→1
分鏡 → 系統
9 張 Redesign 分鏡共用一套 Design Tokens，最後長成同一個網站。

1→8
資料 → 後台
8 個模型同時服務前台查詢與 Django Admin；資料是產品的一部分。

12
測試 → 信心
12 個測試讓每次調整都能快速確認頁面、查詢、會員與後台沒有壞。

三個學習

1. **資料模型決定 UI 的彈性**
   先把 Location、Daily、Hourly、Alert 的關聯想清楚，篩選與詳情頁才有辦法自然長出來。
2. **設計系統必須進到程式碼**
   只做 mockup 不夠；把色彩、字級、間距與元件變成 CSS 與 template，才不會每頁都走鐘。
3. **部署是最後一哩路，也是驗收的一部分**
   Neon 連線、環境變數、collectstatic、health check 與測試帳號，少一個都無法 demo。

限制與下一步

限制：氣象資料為模擬資料，不是即時 NWS API；影響範圍圖為示意；目前以美國 12 個城市為主。

下一步：接 NWS / OpenWeather API、加入真正的通知寄送、PWA 離線快取、更多城市與無障礙稽核。

可維護性：seed_demo 可重建資料；管理員可透過後台新增城市與警報，不需改程式。

Real API　Notifications　PWA　A11y audit

> “ **好的介面不是把資料變少，而是把決定變簡單。**
> Skyline Weather 從 8 個問題出發，最後留下 5 個能回答問題的頁面。

第 8 組
2026.11.01

— 謝謝老師與助教 —　Skyline Weather • 2026 使用者介面設計期中專題

交付資訊

前後台連結、測試帳號、組員分工、AI 對話截圖索引與檔案位置。

2026.11.01

23:59 TronClass

連結與帳號

前台	**https://skyline-weather-gqoc.onrender.com/**
後台	**https://skyline-weather-gqoc.onrender.com/admin/**
GitHub	**https://github.com/0x4b4149/UI_Design_Mid**
會員測試	**demo / Demo1234!**
管理員	**admin / Admin1234!**
組別	**第 8 組**
組員	**組員：_____（學號：_____）**

Demo 建議流程：開啟前台 → 以 demo 登入 → 首頁切換預設城市 → 城市總覽查 Seattle → 收藏 → 警報中心看 Severe → 後台修改一則警報 → 回到前台確認同步。

AI 對話截圖索引（10 組）

01 任務範圍與報告架構
02 原網站 6 張截圖
03 連結／字級／Lighthouse
04 8 項 Bad UI 問題
05 Persona / Story / Scenario
06 三條 User Flow
07 Design System + 9 分鏡
08 前期報告 PDF
09 期中規範
10 全端實作

截圖需包含實際 Prompt 與 AI 回覆，不可只貼整理後文字；前期 1–8 於 2026-10-18 對話，本期 9–10 於 2026-10-31 對話。

檔案位置

程式	**Django project（forecast app）**
資料	**seed_demo.py • 12 cities / 8 alerts**
截圖	**docs/screenshots/ 01–18**
Prompt	**docs/AI_PROMPTS.md**
部署	**docs/DEPLOY.md • render.yaml**
報告	**本 PDF • 另附 PPTX**

驗收流程 • 5 分鐘

demo ready

1 • 前台
開啟首頁與城市總覽

2 • 查詢
搜尋 Seattle / 溫度區間

3 • 登入
demo 收藏與偏好

4 • 警報
Severe 詳情與行動

5 • 後台
修改資料 → 前台同步

資料為課堂展示用模擬資料，依 NWS 公開欄位結構整理；非美國國家氣象局官方網站。